Blueprint de Integración Wompi

Arquitectura Segura y Validación Anti-Spoofing para SaaS Financiero.
Desarrollado por: AurexCode

> **El Problema Actual:** La gestión manual de pagos a través de comprobantes de WhatsApp o capturas de pantalla genera fricción comercial, errores humanos en contabilidad y un cuello de botella en la escalabilidad de cualquier empresa B2B.

1. Visión Arquitectónica de Microservicios

Para garantizar la integridad transaccional, en AurexCode implementamos una arquitectura desacoplada utilizando **Next.js (App Router)** y **Supabase (PostgreSQL)**. El flujo se divide en dos dominios estrictos:

- **Aplicación Principal (Vitrina):** Gestiona el marketing y la adquisición. Solo genera "Intenciones de Compra" (Draft Orders) con estado *PENDING*.
- **Micro-frontend Financiero:** Un entorno aislado que recibe el código de referencia, bloquea el precio en el servidor e inicializa el widget de Wompi, evitando manipulaciones en el navegador (DOM).

2. Prevención de Fraude: Firmas SHA-256

La vulnerabilidad más común en integraciones web es la alteración de precios por parte de usuarios maliciosos inspeccionando el código HTML. Para prevenir esto, generamos una firma criptográfica en el servidor (Backend) antes de renderizar el botón de pago.

El string de validación se construye concatenando: `Referencia + Monto en Centavos + Moneda + Secreto de Integridad`. Si un atacante modifica el valor en el navegador, el hash inyectado ya no coincidirá, y la pasarela de Wompi rechazará la transacción instantáneamente.

3. El Flujo de Webhooks y Anti-Spoofing

La consolidación del pago no depende del navegador del cliente, sino de una comunicación silenciosa de servidor a servidor.



Validación del Checksum

Para evitar que un atacante simule ser Wompi enviando peticiones falsas (spoofing) al endpoint `/api/webhooks`, nuestro servidor reconstruye el hash utilizando el Secreto de Eventos y lo compara milimétricamente con el proporcionado en el payload.

```
const signature = payload.signature; const properties = signature.properties; const stringToHash = properties.map(prop => { return payload.data[prop]; }).join("") + timestamp + WOMPI_EVENTS_SECRET; const localHash = crypto.createHash("sha256").update(stringToHash).digest("hex"); if (localHash !== signature.checksum) { throw new Error("Firma de webhook inválida. Intento de Spoofing."); }
```

4. Row Level Security (RLS) Bypass

La base de datos bloquea operaciones de lectura/escritura públicas. Durante el Webhook, instanciamos Supabase con una **Service Role Key**. Esto nos permite actualizar el estado de la factura de *PENDING* a *APPROVED* en un entorno de privilegios máximos, sin exponer credenciales al cliente.